



J D College Of Engineering & Management

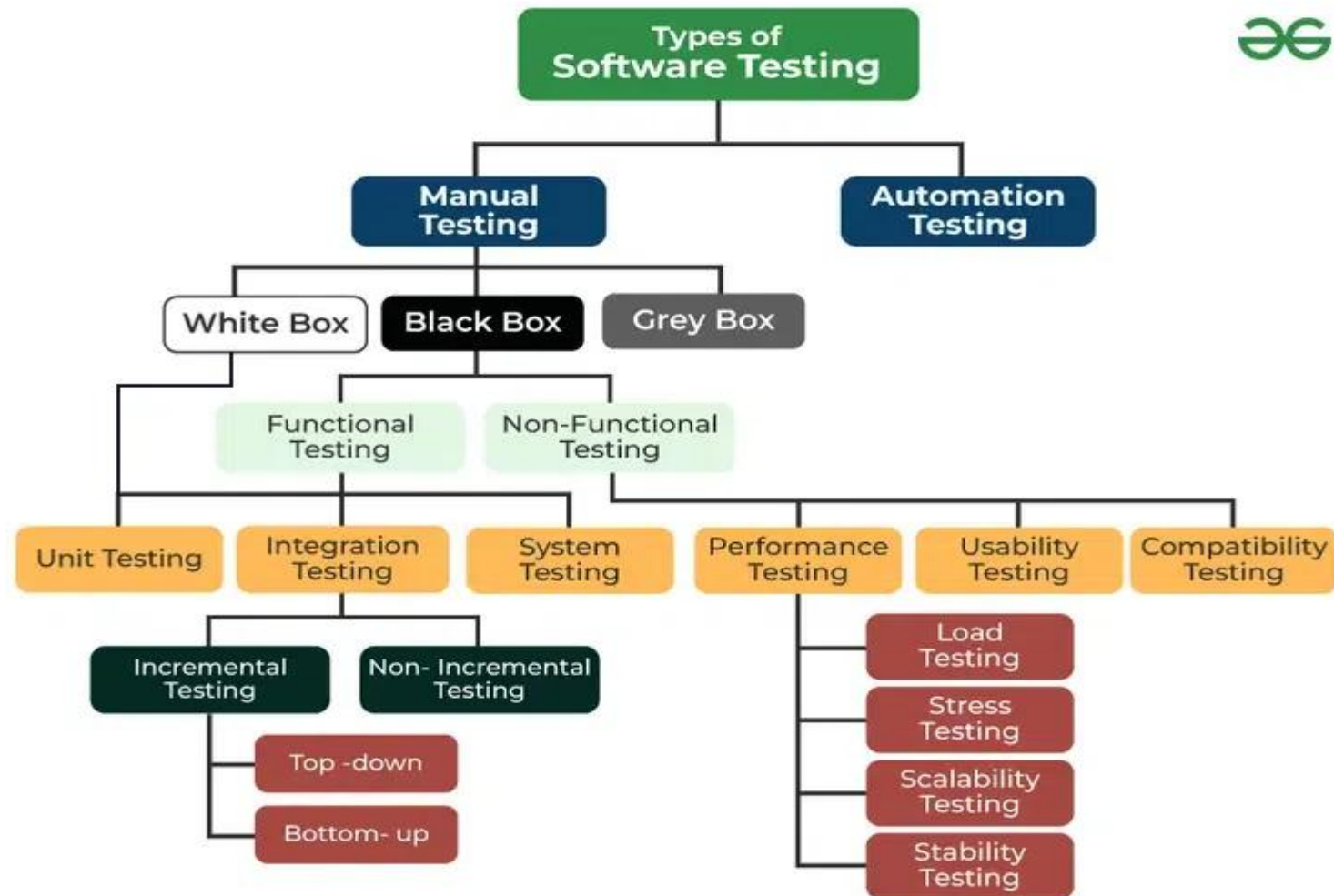
Unit-5

Software Testing Categories

Dr. A. R. Welekar

Associate Professor(CSE)

❖ Types of Software Testing:



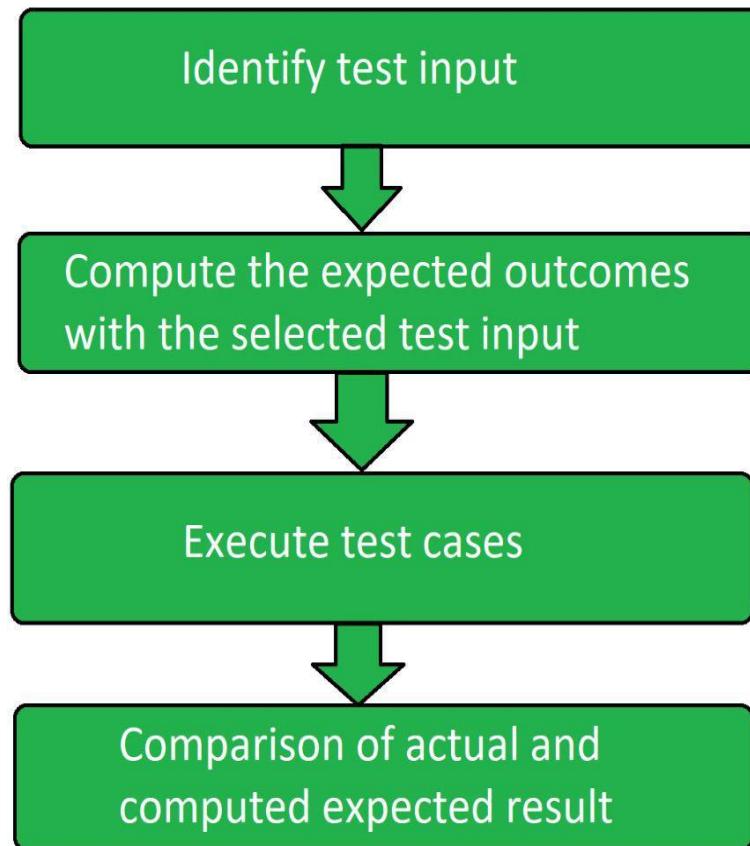
❖ Functional Testing:

- ❑ Functional Testing is defined as a Type of Software Testing that verifies that each function of the Software Application works in conformance with the requirements and specifications.
- ❑ This testing is not concerned with the source code of the application. Each functionality of the software application is tested by providing appropriate test input, expecting the output, and comparing the actual output with the expected output.
- ❑ This testing focuses on checking the user interface, APIs, Database, Security, Client or Server Application, and functionality of the Application Under Test. Functional testing can be performed manually or through automation, depending on the needs of the project.
- ❑ Functional testing mainly involves black box testing and can be done manually or using automation.

❖ Functional Testing:

❑ Functional Testing Process:

Functional testing follows four steps to check if your app or software works as expected:



❖ Functional Testing:

❑ Functional Testing Process:

➤ Step 1. Identify test input

- ✓ The first step in functional testing is to identify the functionality that needs to be tested. This involves determining which core features of the application, such as login, registration, or payment processing, should be tested.
- ✓ It also includes testing usability functions like buttons, forms, and navigation links.
- ✓ Additionally, error conditions must be considered, such as invalid inputs or scenarios where the system fails.

❖ Functional Testing:

❑ Functional Testing Process:

➤ Step 2. Compute expected outcomes

- ✓ Once the test inputs are identified, the next step is to calculate the expected outcomes based on the functional specifications of the system.
- ✓ Here, you define what should happen for each test case based on the input data.
- ✓ For instance, when entering a valid username and password, the expected output could be that the user is successfully redirected to the homepage.
- ✓ On the other hand, for invalid login attempts (like an incorrect password), the expected outcome might be an error message prompting the user to try again.

❖ Functional Testing:

❑ Functional Testing Process:

➤ Step 3. Execute test cases

- ✓ After defining the expected outcomes, the next step is to execute the test cases using the identified inputs. This is where you run the tests and see how the system behaves.
- ✓ Test cases can be executed manually or using automation tools such as Selenium, JUnit, or TestNG.

➤ Step 4. Compare the actual and expected output

- ✓ In the final step of functional testing, you compare the actual output of the system to the expected output.
- ✓ If the system's response matches what was expected, then the functionality is confirmed to be working correctly. This step ensures that the system functions as expected, providing the correct results under various conditions.

❖ Functional Testing:

❑ Type of Functional Testing Techniques:

- **System Testing:** It is a type of testing that is performed on a completely integrated system to evaluate the compliance of the system with the corresponding requirements. In system testing, integration testing passed components are taken as input.
- **Regression testing:** It is done to make sure that the code changes do not affect the existing functionality and the features of the application. It concentrates on whether all parts are working or not.

❖ System Testing:

- ❑ System Testing is carried out on the whole system in the context of either system requirement specifications or functional requirement specifications or the context of both. System testing tests the design and behavior of the system and also the expectations of the customer.

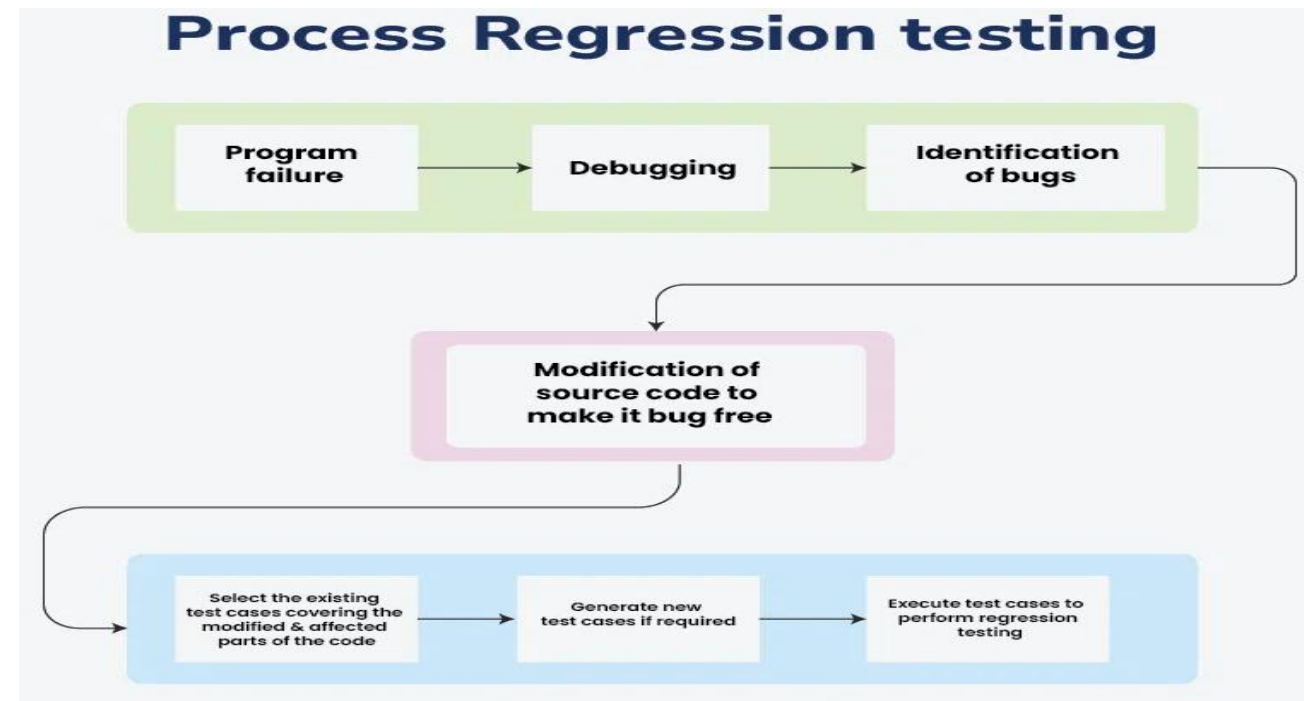
- ❑ **System Testing Process:**

System Testing is performed in the following steps:

- ❑ Test Environment Setup: Create testing environment for the better quality testing.
- ❑ Create Test Case: Generate test case for the testing process.
- ❑ Create Test Data: Generate the data that is to be tested.
- ❑ Execute Test Case: After the generation of the test case and the test data, test cases are executed.
- ❑ Defect Reporting: Defects in the system are detected.
- ❑ Regression Testing: It is carried out to test the side effects of the testing process.
- ❑ Log Defects: Defects are fixed in this step.
- ❑ Retest: If the test is not successful then again test is performed.

❖ Regression Testing:

- ❑ Regression Testing involves re-executing a previously created test suite to verify that recent code changes haven't caused new issues. This verifies that updates, bug fixes, or enhancements do not break the functionality of the application.
- ❑ **Regression Testing Process:**
Regression Testing is performed in the following steps:



❖ Regression Testing:

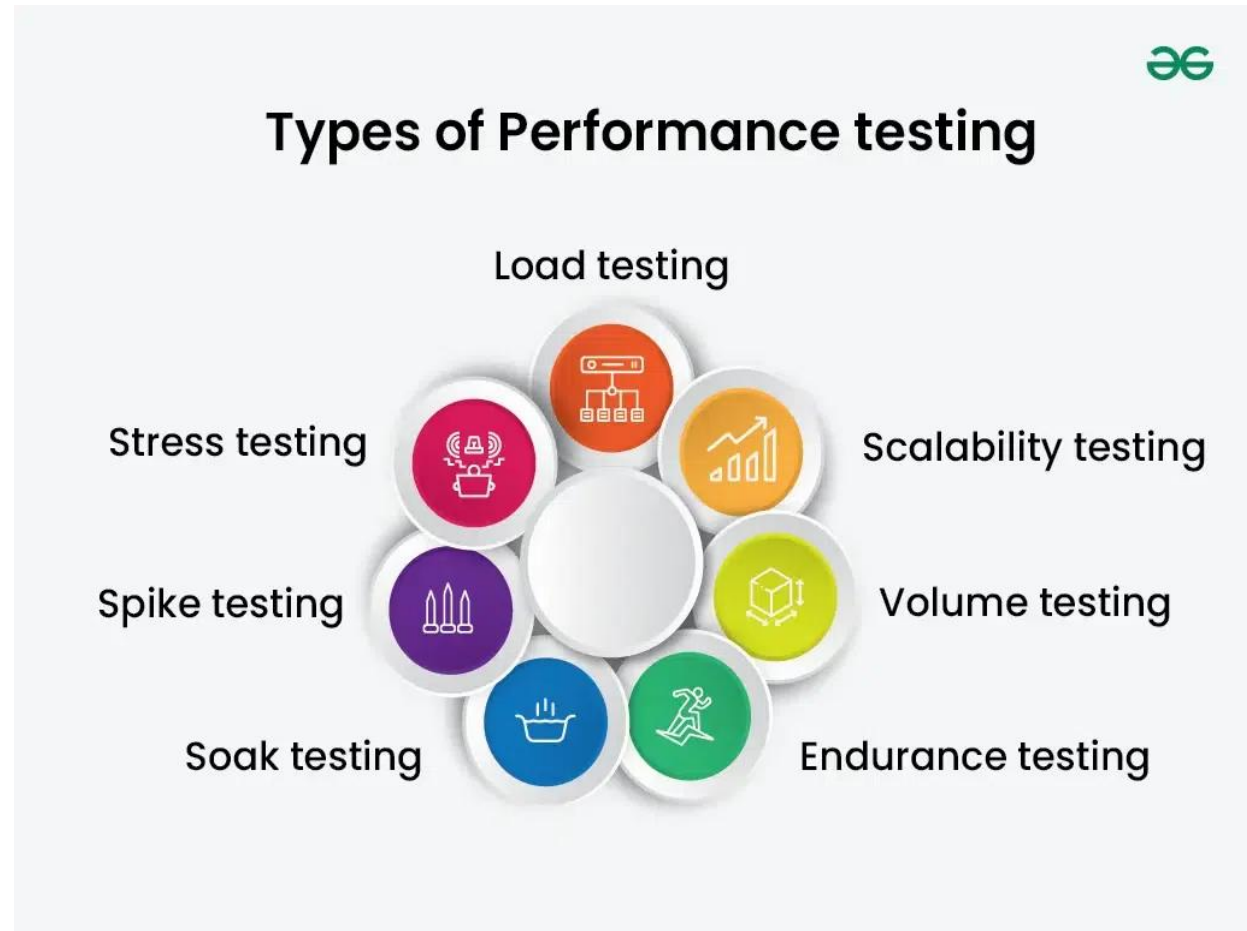
- ❑ **Identify Code Changes:** Analyze the source code to determine which areas have been modified, such as new features, bug fixes, or optimizations.
- ❑ **Debug and Fix Failures:** If existing test cases fail due to changes, debug the code to identify and resolve defects.
- ❑ **Modify Code:** Apply necessary updates to the code to incorporate changes or fixes.
- ❑ **Select Test Cases:** Choose relevant test cases from the existing test suite that cover modified and affected areas. Add new test cases if needed to address new functionality.
- ❑ **Execute Regression Tests:** Run the selected test cases, either manually or using automated tools, to verify system behavior.
- ❑ **Analyze Results:** Review test outcomes to identify regressions, document issues, and recommend fixes.
- ❑ **Retest as Needed:** If defects are found, fix them and re-run tests to confirm resolution.

❖ Performance Testing:

- ❑ Performance Testing is a type of software testing that ensures software applications perform properly under their expected workload. It is a testing technique carried out to determine system performance in terms of sensitivity, reactivity, and stability under a particular workload.
- ❑ Performance testing is a type of software testing that focuses on evaluating the performance and scalability of a system or application.
- ❑ Performance testing aims to identify bottlenecks, measure system performance under various loads and conditions, and ensure that the system can handle the expected number of users or transactions.

❖ Performance Testing:

➤ Types of performance testing:



❖ Performance Testing:

➤ Types of performance testing:

❑ 1. Load testing

Load testing simulates a real-world load on the system to see how it performs under stress. It helps identify bottlenecks and determine the maximum number of users or transactions the system can handle. It checks the product's ability to perform under anticipated user loads. The objective is to identify performance congestion before the software product is launched in the market.

❑ 2. Stress testing

Stress testing is a type of load testing that tests the system's ability to handle a high load above normal usage levels. It helps identify the breaking point of the system and any potential issues that may occur under heavy load conditions. It involves testing a product under extreme workloads to see whether it handles high traffic or not. The objective is to identify the breaking point of a software product.

❖ Performance Testing:

➤ Types of performance testing:

❑ 3. Scalability testing

In Scalability testing , the software application's effectiveness is determined by scaling up to support an increase in user load. It helps in planning capacity additions to your software system.

❖ Load Testing:

- ❑ Load testing is a type of Performance Testing that determines the performance of a system, software product, or software application under real-life-based load conditions.
- ❑ Load testing determines the behavior of the application when multiple users use it at the same time. It is the response of the system measured under varying load conditions.
- ❑ During load testing, various scenarios are simulated to test the system's behavior under different load conditions. This can include simulating a high number of concurrent users, simulating numerous requests, and simulating heavy network traffic.
- ❑ The system's performance is then measured and analyzed to identify any bottlenecks or issues that may occur.

❖ Load Testing:

➤ Objectives of Load Testing

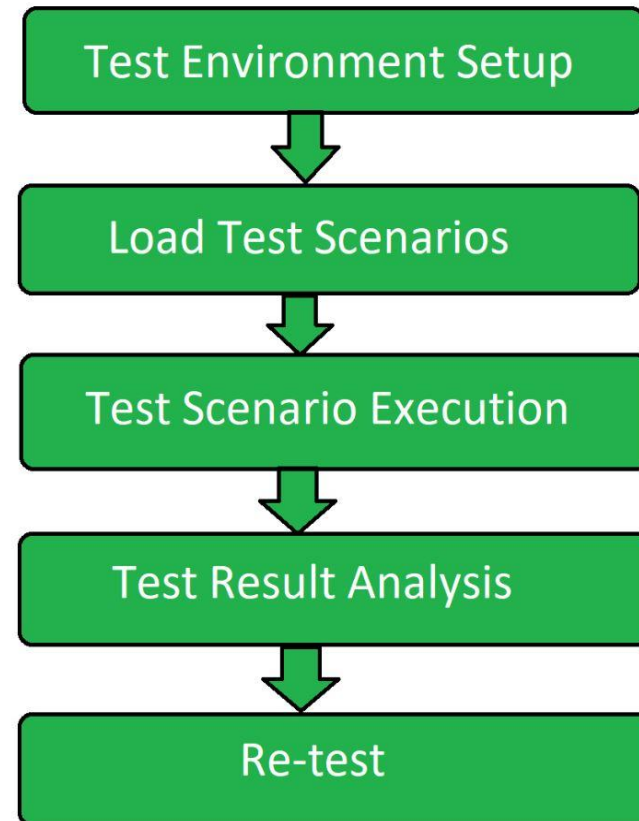
- ❑ **Evaluation of Scalability:** Assess the system's ability to handle growing user and transaction demands. Find the point at which the system begins to function badly.
- ❑ **Planning for Capacity:** Describe the system's ability to accommodate anticipated future increases in the number of users, transactions and volume of data. Making well-informed decisions regarding infrastructure upgrades is made easier by this.
- ❑ **Determine bottlenecks:** Identify and localize bottlenecks in the application or infrastructure's performance. Finding the places where the system's performance can suffer under load is part of this.
- ❑ **Analysis of Response Time:** For crucial transactions and user interactions, track and evaluate response times. Make that the system responds to changes in load with reasonable response times.
- ❑ **Finding Memory Leaks:** Find and fix memory leaks that may eventually cause a decline in performance. Make sure the programme doesn't use up too many resources when it's running.

❖ Load Testing:

➤ Load Testing Process

- ❑ **Test Environment Setup:** Firstly create a dedicated test environment setup for performing the load testing. It ensures that testing would be done in a proper way.
- ❑ **Load Test Scenario:** In second step load test scenarios are created. Then load testing transactions are determined for an application and data is prepared for each transaction.
- ❑ **Test Scenario Execution:** Load test scenarios that were created in previous step are now executed. Different measurements and metrics are gathered to collect the information.
- ❑ **Test Result Analysis:** Results of the testing performed is analyzed and various recommendations are made.
- ❑ **Re-test:** If the test is failed then the test is performed again in order to get the result in correct way.

❖ Load Testing:



❖ Load Testing:

➤ Metrics of Load Testing

Metrics are used in knowing the performance of load testing under different circumstances. It tells how accurately the load testing is working under different test cases. It is usually carried out after the preparation of load test scripts/cases. There are many metrics to evaluate the load testing. Some of them are listed below.

❑ 1. Average Response Time

It tells the average time taken to respond to the request generated by the clients or customers or users. It also shows the speed of the application depending upon the time taken to respond to the all requests generated.

❑ 2. Error Rate

The Error Rate is mentioned in terms of percentage denotes the number of errors occurred during the requests to the total number of requests. These errors are usually raised when the application is no longer handling the request at the given time or for some other technical problems. It makes the application less efficient when the error rate keeps on increasing.

❖ Load Testing:

➤ Metrics of Load Testing

❑ 3. Throughput

This metric is used in knowing the range of bandwidth consumed during the load scripts or tests and it is also used in knowing the amount of data which is being used for checking the request that flows between the user server and application main server. It is measured in kilobytes per second.

❑ 4. Requests Per Second

It tells that how many requests are being generated to the application server per second. The requests could be anything like requesting of images, documents, web pages, articles or any other resources.

❖ Load Testing:

➤ Metrics of Load Testing

❑ 5. Concurrent Users

This metric is used to take the count of the users who are actively present at the particular time or at any time. It just keeps track of count those who are visiting the application at any time without raising any request in the application. From this, we can easily know that at which time the high number of users are visiting the application or website.

❑ 6. Peak Response Time

Peak Response Time measures the time taken to handle the request. It also helps in finding the duration of the peak time(longest time) at which the request and response cycle is handled and finding that which resource is taking longer time to respond the request.

❖ Stress Testing:

➤ What is Stress Testing?

- ❑ Stress testing is defined as types of software testing that verifies the stability and reliability of the system. This test particularly determines the system's robustness and error handling under the burden of some load conditions.
- ❑ It tests beyond the normal operating point and analyses how the system works under extreme conditions. Stress testing is performed to ensure that the system does not crash under crunch situations. Stress testing is also known as Endurance Testing or Torture Testing .
- ❑ The process includes assessing the system's response to heavy loads, identifying weaknesses, and ensuring it remains stable under stress.

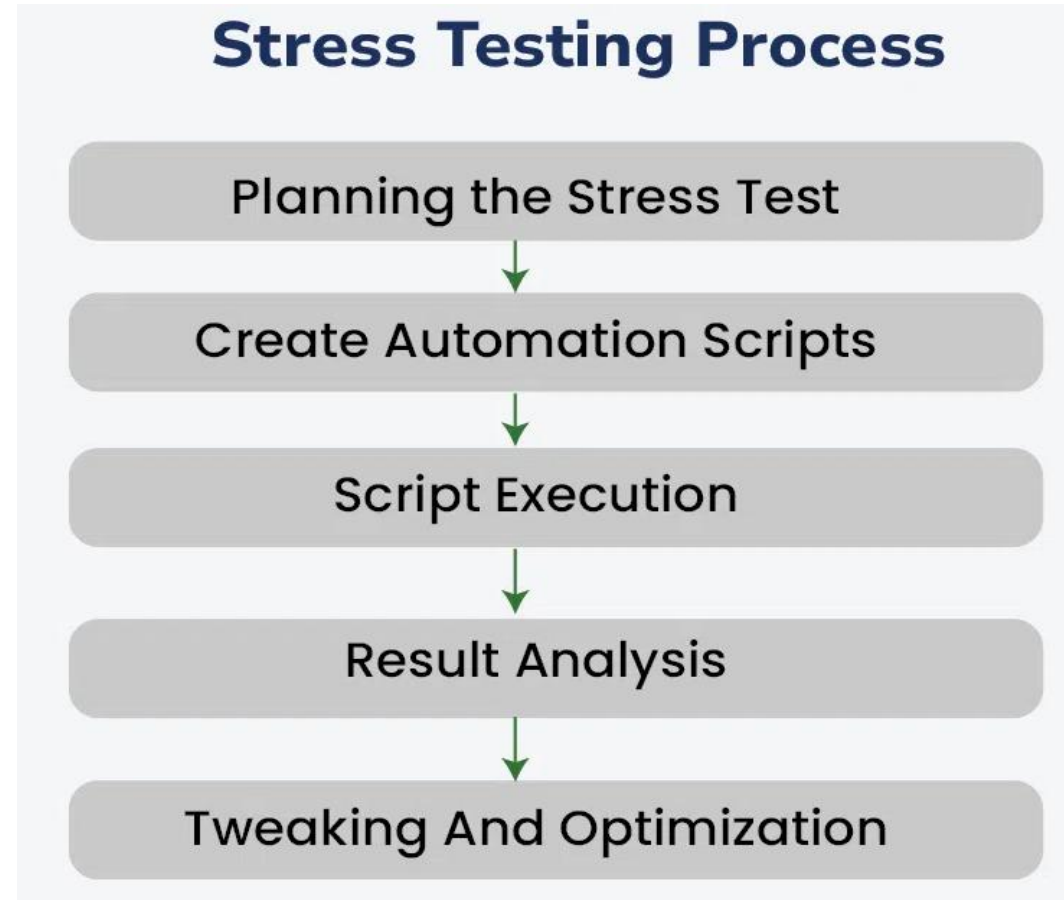
❖ Stress Testing:

➤ Purpose of Stress Testing

- ❑ **Analyze the behavior of the application after failure:** The purpose of stress testing is to analyze the behavior of the application after failure and the software should display the appropriate error messages while it is under extreme conditions.
- ❑ **System recovers after failure:** Stress testing aims to make sure that there are plans for recovering the system to the working state so that the system recovers after failure.
- ❑ **Uncover Hardware issues:** Stress testing helps to uncover hardware issues and data corruption issues.
- ❑ **Uncover Security Weakness:** Stress testing helps to uncover the security vulnerabilities that may enter into the system during the constant peak load and compromise the system.
- ❑ **Ensures data integrity:** Stress testing helps to determine the application's data integrity throughout the extreme load, which means that the data should be in a dependable state even after a failure.

❖ Stress Testing:

➤ Stress Testing Process



❖ Stress Testing:

➤ Stress Testing Process

1. Planning the stress test

- ❑ This step involves gathering the system data, analyzing the system, and defining the stress test goals.
- ❑ Gathering System Data : For a web application, collect details about server specifications, database configurations, network bandwidth, and current traffic patterns.
- ❑ Analyzing the System : Evaluate how the web application performs during peak traffic times, such as during a sales event. Identify any existing issues like slow page loads or database timeouts.
- ❑ Defining Stress Test Goals : Set goals such as ensuring the application can handle a traffic spike of 10,000 users per minute without crashing and maintaining response times below 2 seconds.

❖ Stress Testing:

➤ Stress Testing Process

2. Create Automation Scripts

- ❑ This step involves creating the stress testing automation scripts and generating the test data for the stress test scenarios.
- ❑ Developing Stress Testing Scripts : Use tools like Apache JMeter or LoadRunner to create scripts that simulate user actions such as logging in, browsing products, and making purchases.
- ❑ Generating Test Data : Create datasets that include various user profiles, product inventories, and transaction records to ensure the test is realistic and comprehensive.

❖ Stress Testing:

➤ Stress Testing Process

3. Script Execution

- ❑ This step involves running the stress test automation scripts and storing the stress test results.
- ❑ Analyzing Test Data : Review the logs to identify performance trends. Look for any significant slowdowns or failures as the load increases.
- ❑ Identifying Bottlenecks : If the application slows down significantly at 5,000 users, investigate the server logs to determine if the issue is related to database queries, server CPU limits, or network bandwidth.

❖ Stress Testing:

➤ Stress Testing Process

4. Result Analysis

- ❑ This phase involves analyzing stress test results and identifying the bottlenecks.
- ❑ Running the Stress Test Scripts : Execute the scripts in a test environment that mirrors the production setup. Gradually increase the number of simulated users to observe how the system handles the load.
- ❑ Storing Test Results : Use logging tools to capture performance data such as server response times, error rates, and resource utilization metrics.

❖ Stress Testing:

➤ Stress Testing Process

5. Tweaking and Optimization

- ❑ This step involves fine-tuning the system and optimizing the code with the goal meet the desired benchmarks.
- ❑ Fine-Tuning the System : Based on the findings, optimize database queries, increase server CPU capacity, or enhance network configurations.
- ❑ Optimizing for Desired Benchmarks : Retest the application to ensure it now handles 10,000 users per minute with response times below 2 seconds.

❖ Scalability Testing:

➤ What is Scalability Testing

- ❑ Scalability Testing is a type of non-functional testing in which the performance of a software application, system, network or process is tested in terms of its capability to scale up or scale down the number of user request load or other such performance attributes. It can be carried out at a hardware, software or database level.
- ❑ Scalability Testing is defined as the ability of a network, system, application, product or a process to perform the function correctly when changes are made in the size or volume of the system to meet a growing need.
- ❑ It ensures that a software product can manage the scheduled increase in user traffic, data volume, transaction counts frequency and many other things. It tests the system, processes or database's ability to meet a growing need.

❖ Scalability Testing:

➤ Objective of Scalability Testing:

- ❑ To determine how the application scales with increasing workload.
- ❑ To determine the user limit for the software product.
- ❑ To determine client-side degradation and end user experience under load.
- ❑ To assess the system's performance under various network circumstances, such as latency and bandwidth fluctuations, in order to guarantee dependable operation in a range of settings.
- ❑ To determine whether the system is capable of withstanding scenarios of high usage, making sure that unexpected spikes in traffic can be accommodated without causing performance issues.
- ❑ To guarantee that the system's scalability prevents performance decline and maintains acceptable response times, both of which improve user experience.

❖ Scalability Testing:

➤ Objective of Scalability Testing:

- ❑ To determine server-side robustness and degradation.
- ❑ To help developers improve the system design or code by pointing out locations that could become bottlenecks when the load grows.
- ❑ To evaluate the effective use of system resources, including CPU, memory and network bandwidth, in relation to the system's increasing load, in order to guarantee resource management.
- ❑ To make that the system satisfies performance criteria and offers a satisfying user experience, assess the system's response time under various loads.

❖ Scalability Testing:

➤ Scalability Testing Attributes:

- ❑ **Response Time:** Response time is the time consumed between the user's request and the application's response. Response time may increase or decrease based on different user load on the application. Basically, the response time of an application decreases as the user load increases. Application having the lesser response time is considered as the higher performance application.
- ❑ **Throughput:** Throughput is the measurement of number of requests processed in a unit time by the application. It differs from one application to another as in web application it is measured in number of user requests processed in a unit time whereas in database application it is measured in number of queries processed in a unit time.

❖ Scalability Testing:

➤ Scalability Testing Attributes:

- ❑ **Performance measurement with number of users:** Depending on the application type, it is always tested for the number of users that it can support without its breakdown or busy standby situation.
- ❑ **Threshold load:** Threshold load is the number of requests or transactions the application can process with desired throughput.
- ❑ **CPU Usage:** CPU Usage is the measurement of the CPU utilization while executing application code instructions. It is basically measured in terms of the unit Megahertz.
- ❑ **Memory Usage:** Memory usage is the measurement of the memory consumed for performing a task by an application. It is basically measured in terms of the unit bytes.
- ❑ **Network Usage:** Network usage is the measurement of the bandwidth consumed by an application under test. It is measured in terms of bytes received per second, frames received per second, segments received and sent per second etc.

❖ Scalability Testing:

➤ Steps of Scalability Testing:

- ☐ Define a process that is repeatable for executing scalability test.
- ☐ Determine the criteria for scalability test.
- ☐ Determine the software tools required to carry out the test.
- ☐ Set the testing environment and configure the hardware required to execute a scalability test.
- ☐ Create and verify visual script.
- ☐ Create and verify the load test scenarios.
- ☐ Execute the test.
- ☐ Evaluate the result.
- ☐ Generate required report.

❖ Documentation Testing:

- Testing documents are prepared at different stages. These documents are discussed as follows.

1. Before Testing:

- Since testing begins with the generation of the test cases. The following documents are required for reference
 - ✓ SRS document - Functional Requirements document.
 - ✓ Test Policy document - It means the product must be tested far before release.
 - ✓ Test Strategy document - It mentions detailed aspects of test the team, responsibility matrix, and rights/responsibilities of the test manager and test engineer.
 - ✓ Traceability Matrix document - This is SDLC document, that is related to the requirements-gathering process. As new requirements come, they are added to this matrix. They can be traced forward and backward. These matrices help testers know the source of the requirement.

❖ Documentation Testing:

2. During Testing:

- While testing is started and is being done, the following documents may be required.
 - ✓ Test Case document - It contains the list of to-be tests. It includes various testing like Unit test plan, Integration test plan, System test plan and Acceptance test plan.
 - ✓ Test description - It is a detailed description of all test cases and procedures for executing them.
 - ✓ Test case report - It contains a test case report resulting from the test.
 - ✓ Test logs - It contains test logs for every test case report.

3. After Testing:

- After testing, only the test summary remains which is a collective analysis of all test reports and logs. The software is released under the version control system if it is ready to launch. It summarizes and concludes whether the software is ready to launch.